Sistemas Distribuídos — Trabalho Prático 2
Transferência de Arquivos Peer-to-Peer

CEFET-MG · 2026/2 · Profa. Michelle Hanne
Alline Santos Ferreira, Pedro Henrique Estevam Vaz de Melo e Rávilla Moreira
Código-fonte: https://github.com/allinesferreira/CEFET-SD_TP2

1. Introdução

Este trabalho implementa um sistema elementar de transferência de arquivos no modelo P2P. Cada peer é um processo que atua ao mesmo tempo como servidor, atendendo pedidos de blocos na sua porta, e como cliente, pedindo os blocos que faltam aos vizinhos configurados. O seeder inicial divide o arquivo em blocos de tamanho fixo; os leechers baixam esses blocos e, assim que um bloco é validado e gravado, passam a oferecê-lo aos outros peers. A implementação foi feita em C++17 sobre a API de sockets POSIX do Linux, com OpenSSL para o SHA-256. Não há tracker: os vizinhos de cada peer são passados na linha de comando. Foram avaliados os 24 cenários da Tabela 1 do enunciado, com 5 repetições cada, além de um cenário com a queda de um vizinho.

2. Arquitetura e decisões de projeto

2.1. Peer simétrico e concorrência

Cada processo recebe seu IP e porta (`--ip`, `--port`) e a lista de vizinhos (`--neighbor`). O servidor roda em uma única thread com sockets não bloqueantes e `poll()`: aceita conexões, acumula os bytes de cada requisição até ela estar completa e envia a resposta conforme o socket aceita escrita. Assim, um cliente lento ou parado não impede o atendimento dos demais, e conexões ociosas por mais de 5 s são fechadas. Em teste, o servidor atendeu 300 conexões simultâneas com apenas duas threads no processo.

O lado cliente usa uma thread por vizinho. Cada uma consulta o mapa de blocos do vizinho e pede os blocos que ele tem e que o peer ainda não possui. Dois vetores guardam o estado local: have marca os blocos gravados e validados e `claimed` os que estão sendo baixados, para que duas threads não peçam o mesmo bloco; ambos são protegidos por um mutex. Um bloco só entra em have depois de gravado, e a partir daí o servidor já o entrega a quem pedir. As threads de download percorrem os índices em uma ordem aleatória própria: com ordem sequencial, leechers que baixam do mesmo vizinho ficavam com os mesmos blocos e quase não trocavam entre si (em uma medição com 4 peers e 20 MB, C recebeu menos de 1% dos blocos de B; com ordem aleatória, cerca de 35%).

2.2. Fragmentação, metadados e montagem

Para um arquivo de F bytes e blocos de B bytes há $N = \lceil F/B \rceil$ blocos; o último pode ser menor. O seeder gera um arquivo de metadados textual (`P2PMETA1`) com nome, tamanho, tamanho do bloco, SHA-256 do arquivo e um SHA-256 por bloco, uma informação por linha. O leecher obtém os metadados de um vizinho, cria um arquivo `.part` com o tamanho final e grava cada bloco direto na sua posição com `pwrite`. O hash de cada bloco recebido é conferido antes de ele ser aceito; um bloco corrompido é descartado e pedido de novo. Ao final, o SHA-256 do arquivo inteiro é comparado com o dos metadados, o `.part` é renomeado e o leecher grava sua cópia dos metadados. Como os metadados vêm de outro peer, o nome não pode conter diretórios e os números precisam estar dentro de limites fixos; metadados fora disso são recusados. Cada peer trava a própria pasta, e um segundo processo que tente usar a mesma pasta se recusa a iniciar.

2.3. Protocolo e monitoramento

Cada requisição usa uma conexão TCP e começa com um comando de 1 byte seguido do endereço de quem pede (IPv4 em 4 bytes e porta em 2), usado nos logs do servidor. Toda resposta é um inteiro de 32 bits com o tamanho do conteúdo, seguido do conteúdo; tamanho zero indica que o pedido não pode ser atendido. Os inteiros estão em network byte order, e as rotinas de envio e recepção repetem send/recv até completar a quantidade esperada de bytes.

Comando	Requisição	Resposta
M (metadados)	M + endereço	texto P2PMETA1
H (mapa de blocos)	H + endereço	N bytes, 1 se o peer tem o bloco
G (bloco)	G + endereço + índice (uint32)	bytes do bloco

Tabela 1. Mensagens do protocolo.

Cada linha do log traz o tempo desde o início do processo. São registrados todos os blocos recebidos e a origem (`RECEBIDO`), todas as requisições atendidas ou negadas e o destino (`ENVIADO`, `NEGADO`), uma linha `STATUS` por segundo com os blocos que o peer possui, vizinhos que deixam de responder e, ao encerrar, um resumo com os blocos recebidos de cada vizinho e enviados para cada um.

3. Metodologia dos testes

O script `scripts/testes.py` combina todos os parâmetros da Tabela 1 do enunciado: 2 e 4 peers, blocos de 1024 e 4096 bytes e arquivos de 10 KB, 20 KB, 1 MB, 5 MB, 10 MB e 20 MB, num total de 24 cenários, cada um repetido 5 vezes com um arquivo aleatório novo (120 execuções). Com 2 peers, A é o seeder e B o leecher. Com 4 peers, B conhece apenas A, C conhece A e B, e D conhece apenas B e C, de modo que todos os blocos de D precisam vir de outros leechers.

Em cada execução o script inicia o seeder, espera ele ficar ativo e inicia os leechers um de cada vez, esperando cada um abrir sua porta antes do próximo. Para cada leecher são verificados: (i) tamanho do arquivo remontado igual ao original; (ii) SHA-256 igual ao original; (iii) arquivo de metadados idêntico ao do seeder; (iv) pelo log, todos os blocos vieram de vizinhos configurados para aquele peer, nenhum faltou e cada bloco aparece no log do vizinho de origem como `ENVIADO` para aquele leecher. Cada peer mede o próprio tempo, do seu início até a gravação do último bloco, o que inclui a obtenção dos metadados; o tempo da execução é o do último leecher a terminar.

No cenário de falha (4 peers, blocos de 1 KB, 20 MB), B é encerrado com `SIGKILL` quando D já tem 25% dos blocos. O teste exige que, depois da queda, D registre B como inacessível e continue recebendo blocos de C, e que C e D terminem com o arquivo íntegro. Tudo rodou em 127.0.0.1, em um notebook com AMD Ryzen 7 5700U (16 threads), 12 GB de RAM e Pop!_OS 24.04, funcionando na bateria, com g++ 13.3 e -02.

4. Resultados

Todas as 120 execuções passaram nas quatro verificações. A Tabela 2 traz o tempo médio e o desvio padrão de cada cenário, e a Figura 1 mostra os mesmos tempos em função do tamanho do arquivo.

Arquivo	2 peers bloco 1024 B	2 peers bloco 4096 B	4 peers bloco 1024 B	4 peers bloco 4096 B
10 KB	4 ± 0	3 ± 0	5 ± 0	7 ± 1
20 KB	5 ± 0	4 ± 1	6 ± 0	7 ± 0
1 MB	148 ± 16	40 ± 1	164 ± 16	51 ± 3
5 MB	705 ± 43	185 ± 2	790 ± 49	218 ± 18
10 MB	1345 ± 33	388 ± 28	1472 ± 55	453 ± 24
20 MB	2660 ± 61	760 ± 46	2860 ± 66	852 ± 72

Tabela 2. Tempo médio ± desvio padrão (ms), 5 execuções por cenário.

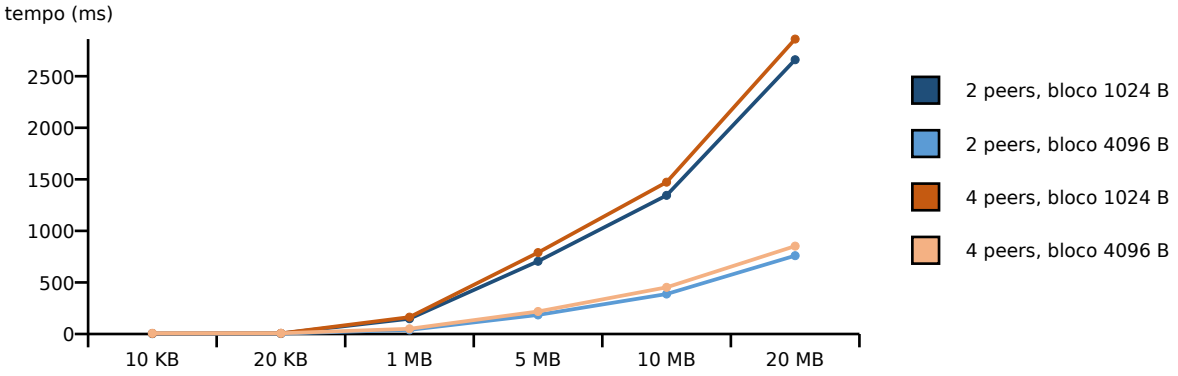


Figura 1. Tempo médio do último leecher por tamanho de arquivo.

Cenário (4 peers)	B (ms)	C (ms)	D (ms)	C ← B	D ← B	D ← C
bloco 1024 B, 5 MB	790	548	550	34%	32%	68%
bloco 1024 B, 10 MB	1472	1008	1015	35%	36%	64%
bloco 1024 B, 20 MB	2860	1898	1953	36%	33%	67%
bloco 4096 B, 5 MB	218	161	165	30%	51%	49%
bloco 4096 B, 10 MB	453	310	325	32%	32%	68%
bloco 4096 B, 20 MB	852	578	591	35%	32%	68%

Tabela 3. Tempo médio de cada leecher (5 execuções) e origem dos blocos recebidos por C e D (primeira execução).

No cenário de falha, B foi encerrado quando D tinha 5234 de 20480 blocos. D registrou B como inacessível, recebeu 15241 blocos de C depois da queda e terminou em 2555 ms, com 2447 blocos vindos de B e 18033 de C; C terminou em 2545 ms, com 17929 blocos de A e 2551 de B. Os dois arquivos ficaram íntegros.

5. Discussão

Tamanho do arquivo. A partir de 1 MB, o tempo cresce quase linearmente com o número de blocos: com 2 peers e blocos de 1 KB, foram 148 ms para 1 MB e 2660 ms para 20 MB. Os arquivos de 10 e 20 KB terminam em poucos milissegundos (no máximo 7 ms), e o tempo é dominado pela troca inicial de metadados.

Tamanho do bloco. Blocos de 4 KB foram de 3,3 a 3,8 vezes mais rápidos nos arquivos de 5 MB em diante (com 2 peers e 20 MB, de 2660 para 760 ms). Cada bloco custa uma conexão TCP, uma requisição e uma verificação de hash; em 127.0.0.1 esse custo fixo pesa mais que o volume de dados, e blocos quatro vezes maiores significam quatro vezes menos requisições.

Número de peers. Com 4 peers, o tempo da execução foi um pouco maior que com 2 (20 MB, blocos de 1 KB: 2860 contra 2660 ms). A Tabela 3 mostra que o mais lento é B: ele só tem A como vizinho, disputa o servidor de A com C e ainda atende C e D. Já C e D, que baixam de dois vizinhos ao mesmo tempo, terminaram em 1898 e 1953 ms, antes do leecher do cenário com 2 peers. Isso mostra o ganho de ter mais fontes e confirma a redistribuição: D recebeu blocos de B e de C, e C recebeu de B entre 30% e 36% dos blocos.

Limitações. Os testes rodaram em uma única máquina, sem a latência, as perdas e o limite de banda de uma rede real; nesse ambiente o custo é dominado por CPU e chamadas de sistema. Só o servidor é não bloqueante: os downloads usam uma thread por vizinho com sockets bloqueantes e timeout de 3 s, o que basta para a concorrência pedida. Os peers descobrem blocos novos por consulta periódica, e não por aviso dos vizinhos, e o endereço de quem pede é declarado pelo próprio cliente, sem autenticação. A implementação aceita só IPv4, um arquivo por processo e não retoma downloads interrompidos.

6. Conclusão

O sistema atende aos requisitos do enunciado: peers simétricos com servidor não bloqueante e downloads concorrentes, fragmentação em blocos de tamanho fixo, formato de metadados e protocolo próprios, registro local dos blocos e remontagem verificada por SHA-256. Os leechers repassam blocos durante o download, como mostram os logs e a Tabela 3, e o sistema se recuperou da queda de um vizinho. Como próximos passos, ficam manter uma conexão por vizinho em vez de uma por requisição, avisar os vizinhos quando um bloco novo chega, testar em máquinas diferentes e permitir retomar um download a partir do `.part`.